

Bursar: Agent-Native HPC

A proposal for campaign-level allocation, low-latency resource leases, and machine-speed back pressure on DOE leadership-class systems

Discussion draft, revision 2 | August 2026

Executive summary

Recommendation. DOE computing facilities should introduce an agent-native resource model in which projects receive campaign-level resource budgets and rate limits, agents obtain short-lived resource leases through a machine API, and millions or billions of fine-grained actions execute inside those leases without becoming scheduler-visible jobs.

The problem is not primarily scheduler throughput. Existing HPC systems were designed around human-scale submission patterns: a person prepares jobs, places a manageable number in a queue, waits, and inspects the results. Agentic scientific workflows invert those assumptions. An agent may make thousands of resource decisions per second, generate enormous contingent task spaces, and require low-latency responses because computation sits inside a closed experimental or reasoning loop. Simply removing job-count limits would expose the batch scheduler to machine-scale demand while leaving the more fundamental scarcity problem unchanged.

Current allocations are necessary but insufficient. An allocation controls cumulative entitlement—how many node- or GPU-hours a project may consume over months—but does not adequately govern instantaneous demand: how many resources the project may consume now, at what response time, and at what opportunity cost to other users. On a fully subscribed machine, those are the variables that determine whether low-latency agent workflows can coexist with large conventional jobs.

The proposed model has six elements:

- **Campaigns, not jobs, become the unit of policy.** A long-lived campaign owns an allocation, scientific objective, identity, budget, and policy envelope; its internal task graph can be generated dynamically. Campaign envelopes nest under a project envelope, so subdividing a project multiplies administrative units but never multiplies entitlement.
- **Resource leases separate global scheduling from task execution.** A campaign broker obtains ordinary allocations from the facility scheduler; Flux, Parsl, Ray, Globus Compute, MPI, or another execution layer handles fine-grained work inside them.
- **Budgets acquire rate and burst controls.** In addition to annual GPU-hours, each campaign receives a guaranteed rate, burst ceiling, burst duration, and optional deadline/latency rights, enforced by token buckets. This is the back-pressure mechanism missing from simple allocation accounting.
- **Service classes make latency an explicit policy choice.** Batch, interactive, agent-burst, deadline, and opportunistic classes carry different allocation multipliers. Immediate access during

congestion costs more than deferrable work, without requiring literal monetary charging. A published congestion index gives agents a machine-readable scarcity signal from day one.

- **Agents receive a resource-intent API.** Instead of submitting millions of jobs, an agent can ask what capacity is available, state a deadline or utility function, and adapt its plan to current scarcity. Candidate tasks are materialized just in time rather than stored individually in the global queue.
- **Envelopes cover more than GPUs.** Machine-speed workloads stress storage, metadata services, networks, and inference services at least as hard as the compute scheduler. The envelope therefore includes I/O and inference dimensions, instrumented from the start and enforced where facility mechanisms permit.

The design principle is simple: make task creation cheap, but make resource consumption explicitly bounded and machine-readable. Agent autonomy should exist inside a resource envelope, not outside resource policy.

A practical pilot does not require a new scheduler. Production facility schedulers—PBS Professional at ALCF (Polaris, Aurora), Slurm at NERSC and LLNL—already provide accounting, QOS/queue classes, concurrent-resource limits, priority, reservations, and preemption. A thin Agent Resource Broker — **Bursar**, named for the institutional officer who meters budgets and, here, bursts — sits above the facility scheduler and adds the missing semantics: campaign leases, token-bucket back pressure, supply signals, a warm capacity pool for seconds-scale lease grants, and composition of multiple scheduler allocations into an elastic campaign pool. The broker defines a scheduler-neutral interface (allocate, release, reserve, preempt, query accounting) with per-facility bindings, so the model is portable across the DOE complex. Agents may generate arbitrary internal work, but the broker cannot acquire resources faster than the campaign policy permits.

1. The policy problem

Leadership computing already manages scarcity. Agentic workloads change the rate at which demand can be expressed: software, not people, may generate resource decisions at machine speed. The problem is how fixed, heavily subscribed infrastructure should admit, shape, and prioritize that demand.

Human-era safeguards are becoming architectural constraints. Job-count limits, submission-rate limits, manual approval, and static job descriptions protect systems built around human submission patterns. They fit poorly when useful computation is discovered dynamically—for example, when an agent chooses the next simulation from the previous result or steers an experiment from inference completed seconds earlier.

Removing those limits is not enough. Removing those limits alone merely creates a larger backlog. The facility must separate (1) how fast work can be described, (2) how fast scarce resources can be consumed, and (3) what latency rights attach to that consumption.

Dimension	Conventional HPC	Agent-native HPC
Policy unit	Job / user / project	Campaign with machine-readable resource envelope

Dimension	Conventional HPC	Agent-native HPC
Demand representation	Finite set of queued jobs	Dynamic task generator / DAG / objective
Scarcity control	Queue + allocation balance + job limits	Budget + sustained rate + burst + service class
Latency	Mostly emergent from queue state	Explicit service attribute with policy cost
Fine-grained execution	Scheduler-visible or user-built pilot layer	First-class execution inside resource lease
Adaptation	Human resubmission / scripts	Continuous agent replanning from supply signals

2. What is genuinely new here—and what is not

Every mechanism in this proposal has an ancestor, and the proposal is stronger for acknowledging them.

- **Pilot jobs and glideins.** HTCondor glideins and the pilot frameworks of high-energy physics (PanDA, DIRAC, GlideinWMS) proved at grid scale that leasing capacity and dispatching fine-grained work inside it removes the global scheduler from the inner loop. Section 3 adopts this pattern directly.
- **Datacenter quota systems.** Google’s Borg demonstrated, in production and at datacenter scale, that per-tenant quota with priority bands—strict guarantees at high priority, deliberately overbooked lower tiers, and preemptible best-effort capacity—can govern machine-generated demand fairly. The envelope design in Sections 4 and 10 is the HPC-allocation translation of that experience.
- **Cloud spot and preemptible markets.** EC2 Spot and preemptible VMs showed that price signals shape elastic demand and that large user communities will engineer around preemption when the discount is real. The opportunistic class and congestion index import that lesson without introducing money.
- **Market-based scheduling research.** Spawn, Tycoon, Bellagio, and related systems explored full auction mechanisms. This proposal deliberately stops short of auctions: fixed class multipliers plus a congestion index capture most of the demand-shaping benefit with far less policy novelty.
- **Flux.** Flux’s hierarchical scheduling already provides lease-like delegation of scheduling authority inside an allocation, and at LLNL much of the broker below could be realized natively in Flux. The broker is proposed as a thin overlay rather than a scheduler replacement because at most facilities the incumbent scheduler (PBS Professional, Slurm) is the entrenched authority for placement, accounting, and security review, and the pilot should not require displacing it.
- **DOE context.** The proposal is a natural resource-control plane for the Integrated Research Infrastructure (IRI) program, whose cross-facility workflows have exactly the closed-loop, latency-sensitive character described here; ExaWorks-lineage substrates (Parsl, RADICAL, Flux) supply the execution layer; and urgent/realtime computing programs are existing precedent for granting temporal privilege, not just capacity.

So what is new? Not leases, not QOS, not pilot jobs. What is new is closing a control loop that has never existed in HPC: *machine-speed demand expression* (agents that can restate their resource intent in milliseconds) coupled to *machine-readable supply signals* (token-bucket state, congestion indices, capacity forecasts). Human users cannot exploit a supply API that changes second by second; agents can. The claim of this proposal is that once both ends of that loop run at machine speed, back pressure stops being a queue-length phenomenon and becomes an input to scientific planning—and that facilities should therefore engineer the loop deliberately rather than let it emerge accidentally. The mechanisms benefit any adaptive high-throughput workflow, agentic or not; agents are simply the first client that can use the loop at full speed.

3. Design principle: schedule envelopes, not actions

The global scheduler should make relatively infrequent decisions about resource ownership; an execution substrate should make high-rate decisions about actions inside those resources. This preserves global policy while removing task counts from the facility scheduler.

The proposed primitive is a resource lease. A lease is a time-bounded grant of resources to a campaign. For the pilot, a lease maps to one or more ordinary scheduler allocations. Elasticity is composed rather than assumed: the broker can add or release allocations as policy and capacity permit, while the execution substrate presents their union as one changing resource pool. This avoids requiring arbitrary live resizing of a running scheduler job.

Scope: composed elasticity fits independent and loosely coupled work. A pool composed from several scheduler allocations spans disjoint node sets with no shared launcher, which suits task-parallel, ensemble, and pipeline workloads—the dominant shape of agentic campaigns. It does not suit a single tightly coupled computation that must gang-schedule N GPUs: such work must request a *single-allocation lease* of that size and pays whatever queue or reservation latency that entails. Campaigns declare which shape they need in the intent API (Section 6), and the pilot’s target workloads are drawn from the independent-task class.

Pilot-job systems already demonstrate the execution separation. Flux, Parsl, Ray, Globus Compute, MPI worker pools, and similar systems can dispatch work at high rates once the facility scheduler is removed from the inner loop. The new component is a campaign broker that gives this pattern explicit allocation, rate/burst, and fairness semantics.

Implementation boundary. The facility scheduler remains the authoritative placement, accounting, priority, reservation, and preemption system. The broker owns only agent-facing policy state that the scheduler does not express cleanly—especially explicit token buckets, supply forecasts, congestion indices, and campaign-level composition of leases. The broker’s scheduler interface is deliberately narrow—allocate, release, reserve, preempt, query accounting—with bindings for PBS Professional (ALCF) and Slurm (NERSC, LLNL); a Flux binding is a natural third.

Global scheduler: “Who owns these 256 GPUs for the next 20 minutes?” Execution layer:
“Which of the next 4.7 million candidate actions should run on them?”

4. Replace one-dimensional allocations with resource envelopes

Today an allocation is primarily a stock of node- or GPU-hours. Agentic workloads require stock *plus* flow controls. A resource envelope should include:

- **Total budget:** cumulative entitlement over the allocation period, in class-weighted credits (Section 5).
- **Guaranteed rate:** capacity the campaign can obtain on demand under normal conditions.
- **Sustained-rate ceiling (R):** maximum long-duration consumption rate; also the token replenishment rate.
- **Burst ceiling:** short-duration maximum above the sustained rate.
- **Burst bucket (B):** how much above-sustained consumption can accumulate before throttling occurs.
- **Service class:** batch, interactive, agent-burst, deadline, or opportunistic.
- **Preemption class:** whether and how quickly resources may be reclaimed.
- **Optional deadline/value declaration:** how utility changes with completion time.

A token-bucket model is a natural implementation. For each campaign, a token bucket governs *resource acquisition*, not task submission. Define the pair (R, B): tokens replenish at the sustained rate R (measured in GPUs, i.e., GPU-minutes per minute) and accumulate up to the bucket capacity B (measured in GPU-minutes). Active leases drain tokens at their held-capacity rate—held, not merely used, so idle holding is inherently costly. A campaign with saved tokens may burst to its ceiling; as tokens drain, it is forced back toward R. Thus agents may generate unlimited task intentions while actual consumption remains bounded before the global queue ever becomes the back-pressure mechanism.

Stock and flow interact by explicit rule. Tokens meter *rate*; the credit budget meters *cumulative* consumption. The rules are: (1) a lease request must clear both the bucket and the remaining budget; (2) an exhausted budget confines the campaign to the opportunistic class until the allocation committee supplements it; (3) an empty bucket confines the campaign to its replenishment rate R; (4) the guaranteed rate is honored even with an empty bucket, provided budget remains—guarantees are promises about availability, and the bucket exists to police bursts, not floors. Envelope parameters must also be mutually consistent: $R \times \text{allocation-period} \geq \text{total budget}$, so a campaign can always physically spend what it was granted.

Illustrative campaign parameter	Value
Annual budget	3,000,000 GPU-hour credits ($\approx$ 342 GPUs continuous average)
Guaranteed rate	128 GPUs
Sustained ceiling R	1,024 GPUs
Burst ceiling	8,192 GPUs
Burst bucket B	240,000 GPU-minutes (4,000 GPU-hours)
Agent-burst multiplier	3 $\times$ base; scaled by congestion index

Illustrative campaign parameter	Value
Opportunistic multiplier	0.25×, fully preemptible

These parameters are self-consistent by construction: at the full burst ceiling the bucket drains at $8,192 - 1,024 = 7,168$ GPUs above sustained, sustaining roughly 33 minutes of maximal burst; at a more typical 2,048-GPU burst it sustains about four hours. Facilities can tune B to the burst duration their science cases actually need; the arithmetic should always be published alongside the envelope so campaigns can plan against it.

The envelope is not compute-only. Machine-speed campaigns can saturate the parallel filesystem, metadata servers, and wide-area network long before they trouble the scheduler—arguably the larger stability risk at a real facility. The envelope therefore carries additional dimensions: aggregate filesystem bandwidth ceiling, metadata-operation rate, storage capacity quota, and WAN egress rate. Enforcement maturity varies: storage capacity quotas are routine; bandwidth and metadata QOS exist on some systems (Lustre QOS, DAOS) and not others. Where the facility cannot enforce, the broker instruments per-campaign consumption and directs the campaign runtime to throttle at the source, with envelope violation treated as a policy event (warning, then lease shrinkage). The pilot instruments all dimensions from day one and enforces the subset the platform supports.

Inference is a first-class envelope dimension. Most agentic loops depend on model inference with persistent, low-latency endpoints—a resource shape (long-lived, fractional, shared) unlike ensemble leases. The envelope includes an inference dimension (requests/s and tokens/s ceilings), satisfiable two ways: a facility-operated shared inference service metered against the campaign budget, or campaign-hosted serving inside a long-lived interactive-class lease. The pilot should exercise both, since where the agent’s own reasoning runs determines the latency floor of the entire closed loop.

5. Make response time an explicit service, not an accidental outcome

Closed-loop agent workflows can be latency-sensitive even when individual computations are small. The facility should expose explicit service classes. In the pilot these map to scheduler QOS/queue classes with different priority, resource limits, accounting factors, and preemption behavior.

Class	Policy objective
Throughput / batch	Scheduler has broad placement freedom; lowest cost per resource-hour; objective is utilization and eventual completion.
Interactive	Small, bounded resource requests with seconds-to-minutes response targets.
Agent burst	Elastic short-lived leases intended for closed-loop workflows; capacity premium reflects scarcity.
Deadline	Campaign declares a completion deadline; scheduler may reserve or reshape capacity to meet it.
Opportunistic	Very low accounting cost but immediately preemptible; ideal for speculative or easily replayed agent work.

Credits price these classes without introducing real money. Define one *credit* as one batch-class GPU-hour. An immediate GPU-hour from the scarce agent-burst pool debits three credits; preemptible use of otherwise idle capacity debits 0.25. Budgets, intent-API spend limits, and accounting reports are all denominated in credits, so “maximum immediate-spend: 18,000 credits” in Section 6 has a precise meaning: 6,000 burst GPU-hours, or proportionally more in cheaper classes.

A minimal congestion signal ships in the pilot, not after it. The adaptive behavior this proposal predicts—agents deferring low-value work when capacity is scarce—requires a scarcity signal worth adapting to. Fixed class multipliers alone are too static. The pilot therefore includes a deliberately simple mechanism: the broker publishes, per class, a congestion index (demanded ÷ available capacity over a trailing window) and maps it through a fixed, published lookup table to a dynamic multiplier bounded between 1× and 3× the base rate. This is not dynamic pricing in any market sense—the table is static policy, set by the facility—but it gives adaptive agents a real signal and gives the pilot’s adaptivity hypothesis (Section 13) a fair test. Full congestion pricing remains future work and is not a pilot prerequisite.

6. Give agents an intent and supply API

Agents should not interact directly with raw scheduler mechanisms. The Agent Resource Broker should expose a small machine API—query supply, request/release lease, report bucket state—and translate approved requests into scheduler allocations, QOS assignments, reservations, and accounting operations.

The supply API should report capacity forecasts, congestion index, service class, preemption risk, expected lease-grant latency by path (Section 7), and token-bucket state (current tokens, replenishment rate, burst ceiling). An intent can state minimum/target resources, coupling shape (independent tasks vs. gang-scheduled), deadline, malleability, maximum credit spend, and accelerator alternatives.

This changes the interaction from “submit and wait” to negotiation. An agent that sees scarce burst tokens or a high congestion multiplier can rank experiments by expected information gain, run the highest-value subset now, and defer the rest. Back pressure then changes scientific planning rather than merely increasing queue length.

Example intent — *Goal: evaluate up to 10,000 candidate simulations. Minimum useful capacity: 64 GPUs. Target: 1,024 GPUs. Coupling: independent tasks. Deadline: 13:30. Maximum immediate-spend: 18,000 credits. Malleable: yes; checkpointable: yes. Policy: maximize expected information gain subject to budget.*

7. Engineering the latency target

Seconds-scale lease grants do not fall out of QOS settings; on a full machine, a fresh scheduler allocation pays queue wait plus job-start overhead—prolog execution, container pulls, node health checks—which can reach minutes even with an empty queue. The broker therefore serves leases through three explicit paths, and the supply API always reports which path a request would take.

- **Warm path.** The broker holds the agent-burst pool as standing, broker-owned scheduler allocations: nodes on which prologs have already run, container images are pre-staged, and the execution substrate's launcher is resident. Granting a lease from the warm pool is broker bookkeeping plus substrate attach—no scheduler queue transit—with a target of $p50 \leq 5 \text{ s}$, $p95 \leq 30 \text{ s}$ while pool capacity is free.
- **Reclaim path.** When the warm pool is fully leased, the broker preempts opportunistic leases. Opportunistic tenancy is contractual: a published grace window (e.g., 60 s to checkpoint) in exchange for the $0.25\times$ rate. Target: $p95 \leq 3 \text{ min}$.
- **Cold path.** Demand beyond warm plus reclaimable capacity goes to the conventional scheduler as new allocations via backfill or reservation—minutes to hours, honestly forecast. Agents see the forecast and can choose to defer instead.

The warm pool's carrying cost is the utilization premium discussed in Section 14; keeping it small (2–5% of the machine) and backfilling it with opportunistic work bounds that cost, and measuring the residual loss is a primary pilot objective.

8. Do not queue a billion tasks

Machine-scale task spaces should be represented intensionally: as generators, DAGs, search policies, or objectives—not as millions or billions of scheduler records. Concrete actions are materialized just in time as leased capacity becomes available.

This avoids control-plane overload and lets campaigns abandon low-value branches as evidence accumulates. A billion possible actions need not become a billion queued jobs.

9. Make elasticity, preemption, and replay standard workload properties

Agent-burst leases should favor malleable, checkpointable, replayable, or idempotent workloads. The broker can compose several scheduler allocations into one campaign pool and release them independently as capacity changes.

A campaign might therefore appear to the runtime as $128 \rightarrow 640 \rightarrow 1,664 \rightarrow 640 \rightarrow 128$ GPUs while the facility scheduler sees several conventional allocations starting and ending. This provides useful campaign elasticity without requiring general live expansion of a running scheduler job. As Section 3 notes, this composed elasticity applies to independent-task pools; gang-scheduled work scales only at single-allocation boundaries.

10. Admission control, overbooking, and gaming

Token buckets bound each campaign individually; fairness is decided in the aggregate, and the aggregate rules must be explicit.

- **Guarantees are never overbooked.** The sum of guaranteed rates across all campaigns must not exceed warm-pool capacity plus contractually reclaimable opportunistic capacity. A guarantee

that cannot be simultaneously honored is not a guarantee; the allocation committee's admission rule enforces this invariant when granting envelopes.

- **Burst ceilings are deliberately overbooked—by a published factor.** Statistical multiplexing is the point of a shared burst pool: the sum of burst ceilings may exceed pool capacity by an overbooking factor ϕ (e.g., 3–5 $\times$), disclosed to all campaigns. Campaigns know that ceiling access is probabilistic above their guarantee.
- **Correlated bursts are arbitrated by a deterministic, published rule.** When simultaneous demand exceeds the pool, the broker admits in order of: service class first (deadline before agent-burst before opportunistic), then weighted-fair-share deficit within class (campaigns furthest below their entitlement over a trailing window first), with token wealth breaking ties. The rule is deterministic and journaled, so every admission decision can be reconstructed and explained afterward—the governance criterion of Section 13.
- **Splitting campaigns must not multiply entitlement.** Envelopes are hierarchical: the allocation committee grants an envelope to a *project*, and campaign buckets nest inside the project bucket. A project that spawns ten campaigns divides its rate and burst capacity among them; the parent bucket caps the sum.
- **Idle holding is doubly discouraged.** Tokens and credits drain against *held* capacity regardless of use, so hoarding is expensive by construction; additionally, leases carry a utilization covenant (e.g., sustained utilization below 50% over 10 minutes triggers broker-initiated shrinkage with notice). Both the price and the covenant are needed: the price deters rational actors, the covenant handles buggy ones.

11. Security, governance, and failure semantics at machine speed

Machine-speed access raises the consequences of compromised credentials, buggy agents, and pathological feedback loops. It therefore requires stronger control-plane semantics:

- Campaign-scoped identities and short-lived credentials rather than unconstrained user credentials embedded in agents, integrated with existing facility identity and per-user accounting requirements rather than bypassing them.
- Hard resource-rate and spending limits enforced below the agent runtime.
- Per-tool and per-destination authorization for storage, networking, instruments, and external services.
- Kill, pause, and quarantine operations that act on the entire campaign—implemented in the facility scheduler's own mechanisms so they work even when the broker is down.
- Complete provenance for lease acquisition, resource consumption, task generation, and policy changes.
- Anomaly detection on request rate, consumption, failure patterns, and unexpected lateral access.

The broker must fail static, not fail open. The broker sits on the critical path and holds bucket and lease state, so its failure semantics are part of the design, not an operational afterthought. Bucket and lease state live in a replicated store with a write-ahead journal. On broker failure, existing leases remain valid until their scheduled expiry—the underlying scheduler allocations simply continue—but no new leases are granted: campaigns degrade to their current holdings, never to unmetered access. On restart, the broker reconciles by replaying its journal against the scheduler's state, and on any divergence the

scheduler is authoritative for placement and accounting while the journal is authoritative for policy state (bucket levels, admission history). Kill/pause/quarantine act through scheduler mechanisms directly and do not depend on broker availability.

The key distinction is *freedom to decide* versus *freedom to consume*: agents may choose scientific actions broadly while infrastructure continuously enforces resource and privilege bounds.

12. A concrete ALCF pilot

ALCF's production systems (Polaris, Aurora) are scheduled by PBS Professional, so the pilot's primary broker binding targets PBS; a Slurm binding is built in parallel to keep the model portable to NERSC and LLNL and to force the scheduler interface to stay honest. Nothing in the model depends on scheduler choice—only the binding does.

The pilot tests the central hypothesis directly: human-scale task limits can be removed without sacrificing fairness or stability if an Agent Resource Broker uses token buckets to bound resource acquisition and the facility scheduler enforces the resulting allocations.

Phase 1 (months 1–3): broker and synthetic load.

1. **Define the campaign resource-envelope schema** — budget, guaranteed rate, (R, B), burst ceiling, service class, preemption class, I/O and inference dimensions, and the hierarchical project/campaign nesting.
2. **Implement Bursar, the Agent Resource Broker**, with the scheduler-neutral interface and the PBS Professional binding (Slurm binding to follow in Phase 2); supply, bucket-state, congestion-index, and lease APIs; warm-pool management; the deterministic admission rule; and journaled, replicated state with fail-static recovery.
3. **Create the agent-burst pool**. Reserve approximately 2–5% of a machine as broker-held warm capacity, backfilled with opportunistic/preemptible work, implemented with existing scheduler QOS/queue, reservation, and preemption mechanisms.
4. **Drive with synthetic campaigns**: adversarial burst generators, correlated-demand storms, idle-hoarding clients, and broker kill/restart drills. Finalize the quantitative thresholds of Section 13 against measured baselines.

Phase 2 (months 4–9): science campaigns and controls.

5. **Select four to six campaigns** with genuine closed-loop behavior: autonomous experimentation, ensemble simulation, inference-plus-simulation loops, adaptive search, and mixed HPC/AI workflows. (A smaller cohort than a full program deserves, chosen deliberately: instrumenting real closed-loop science is the pilot's costliest activity.)
6. **Assign resource envelopes**. Keep existing project allocations, but assign each campaign a token bucket and service class. The broker meters lease acquisition; the scheduler enforces the approved concurrent resources and accounting. Tokens do not meter internal task submission.
7. **Run existing task substrates inside leases**. Use at least two of Flux, Parsl, Globus Compute, Ray, or MPI pools inside broker-acquired allocations. Permit 10^6 + internal actions without

exposing them individually to the scheduler. Exercise both inference-provisioning modes (shared service and in-lease serving).

8. **Instrument everything.** Measure request-to-start latency by path, token depletion/replenishment, lease utilization, scheduler and filesystem control-plane load, task throughput, preemption cost, allocation burn rate, congestion-index dynamics, I/O envelope consumption, and scientific time-to-decision.
9. **Compare against controls.** Run conventional batch controls and two agent-native conditions: naive agents that simply request capacity, and adaptive agents that observe bucket, congestion, and forecast state and change their plans. Run multiple campaigns concurrently to test isolation and fairness under machine-generated correlated bursts.

13. Success criteria and policy questions

Success requires more than high task rate. The pilot must show that explicit rate/burst control permits low-latency agent work while protecting other campaigns and maintaining acceptable utilization. The thresholds below are initial targets, to be finalized against Phase 1 baselines—but the decision rule in Section 15 binds to them, whatever their final values.

Question	Measure and initial target
Responsiveness	Lease-grant latency, warm path: $p_{50} \leq 5$ s, $p_{95} \leq 60$ s. Reclaim path: $p_{95} \leq 3$ min. Forecast accuracy on cold path within $2\times$ actual.
Isolation	Under adversarial synthetic burst load, unaffected campaigns' p_{95} lease latency degrades $\leq 10\%$; broker API $p_{99} \leq 1$ s; no measurable filesystem/control-plane degradation for non-participants.
Fairness	Each campaign's realized share within 10% of its weighted-fair entitlement over rolling 24-hour windows; every admission decision reconstructible from the journal.
Utilization	Net machine utilization loss attributable to the warm pool ≤ 3 percentage points after opportunistic backfill recovery.
Scientific productivity	Time-to-decision for closed-loop campaigns improves $\geq 2\times$ versus their batch controls.
Adaptivity	Adaptive agents achieve $\geq 20\%$ lower credit cost per unit of scientific output than naive agents, with plan changes demonstrably correlated to congestion-index movement.
Governance	Facility staff can explain, from journal evidence alone, why any campaign received immediate capacity while another waited.

14. Policy implications

Utilization should not be the sole operational proxy for success: a system at 99% occupancy can have terrible response time. If modest reserved headroom yields substantially faster autonomous discovery, some utilization premium may be rational—and the pilot puts a number on both sides of that trade.

Allocation committees would allocate not only resource quantity but *temporal privilege*—guaranteed rates, burst rights, or deadline service. Reservations and urgent queues already do this implicitly; the proposal makes it explicit, machine-enforceable, and (via the admission journal) auditable.

Fairness should mean intelligible resource rules, not equal productivity. A group that uses its allocation more intelligently may accomplish more; it should not consume everyone else’s instantaneous capacity simply because its agents can ask faster.

More capacity remains valuable but does not remove scarcity if useful demand grows faster. Agentic computing therefore strengthens both the case for expanded capacity and the need for admission control, congestion signals, and machine-readable budgets.

15. Decision requested

Authorize a nine-month, two-phase ALCF design-and-pilot effort for an agent-native resource service with the following scope:

- Define a campaign resource-envelope schema covering total budget, token parameters (R, B), burst ceiling, service class, preemption class, I/O dimensions, inference dimensions, and project-level nesting.
- Implement Bursar, a thin Agent Resource Broker with a scheduler-neutral interface, a PBS Professional binding for ALCF and a Slurm binding for portability; supply, bucket-state, congestion-index, and lease APIs; warm-pool lease paths with explicit latency targets; a deterministic, journaled admission rule; and fail-static recovery semantics.
- Establish an agent-burst warm pool of 2–5% of a machine using existing scheduler mechanisms, backfilled with opportunistic/preemptible reclamation.
- Provide at least two inner execution substrates capable of high-rate task dispatch within a lease, and both inference-provisioning modes.
- Pilot with four to six concurrent scientific campaigns, including naive and resource-adaptive agents, plus conventional batch controls, after a synthetic-load hardening phase.
- Report quantitative results against the Section 13 thresholds and recommend whether to expand, modify, or abandon the model.